

DESAFIO PRÁTICO | STRINGS

Sequências, pesquisa, transformação e registros

Tempo sugerido: 40 minutos | Sem executar código | Justifica as respostas

Nome: _____ Turma: _____ Data: _____

1 ÍNDICES E FATIAS

```
texto = 'Python'
```

Indica os resultados:

```
texto[0]      _____
texto[-1]     _____
texto[1:4]    _____
texto[::-1]   _____
texto[10:]    _____
```

Em que difere texto[10] de texto[10:]?

2 NORMALIZAR E PESQUISAR

```
entrada = ' Ana Silva '
nome = entrada.strip().lower()
```

a) Qual é o valor de nome?

b) Avalia as expressões:

```
'ana' in nome
nome.find('Silva')
nome.count('a')
```

c) entrada foi modificada? Justifica.

3 CORRIGE A ALTERAÇÃO

```
texto = 'Gato'
texto[0] = 'R'
```

a) Que erro ocorre e porquê?

b) Cria uma nova string Rato usando uma fatia.

c) Que resultados têm estas expressões?

```
'12' + '3'
int('12') + int('3')
```

4 DIVIDIR E CONVERTER

```
linha = '101;Ana;16.5'
campos = linha.split(';')
```

a) Escreve a lista campos.

b) Constrói o tuplo (101, 'Ana', 16.5), com int, str e float.

c) Antes de aceder aos três campos, que verificação deves fazer?

5 VALIDAR UM IDENTIFICADOR

Após strip(), um identificador deve ter 5 a 10 caracteres, apenas letras e algarismos, e começar por letra. São regras desta aplicação.

a) Classifica Ana12, 12Ana, Ana-1 e Ana.

b) Explica por que se deve testar o comprimento antes de aceder a texto[0].

c) Indica os métodos de string úteis para estas regras.

6 DESAFIO INTEGRADOR

Cria **contar_palavras(texto)**: converte para minúsculas, divide com split() e devolve o número de palavras. Considera palavra cada token separado por espaços em branco.

a) Escreve a função.

b) Testa com 'Python é útil' e com ''.

c) Este método remove pontuação?

DESAFIO PRÁTICO | STRINGS

Sequências, pesquisa, transformação e registros

SOLUÇÕES

Compara os resultados e verifica o raciocínio.

1 ÍNDICES E FATIAS

```
texto[0]    -> 'P'
texto[-1]   -> 'n'
texto[1:4]   -> 'yth'
texto[: -1]  -> 'nohtyP'
texto[10:]   -> ''
```

texto[10] provoca **IndexError**. Numa fatia, um limite além do tamanho não provoca esse erro; neste caso, a fatia é vazia.

2 NORMALIZAR E PESQUISAR

- a) nome vale **'ana silva'**.
- b) Resultados: **True, -1, 3**. find distingue maiúsculas de minúsculas. Como nome está em minúsculas, a substring Silva não é encontrada.
- c) Não. As strings são imutáveis. Os métodos devolvem novas strings e entrada mantém os espaços e as maiúsculas originais.

3 CORRIGE A ALTERAÇÃO

a) Ocorre **TypeError**, porque não se pode substituir um carácter de uma string.

```
novo = 'R' + texto[1:]
```

- b) novo vale Rato. texto continua a valer Gato.
- c) A primeira expressão concatena e devolve **'123'**. A segunda converte para inteiros e devolve **15**.

4 DIVIDIR E CONVERTER

```
campos = ['101', 'Ana', '16.5']
aluno = (int(campos[0]),
         campos[1],
         float(campos[2]))
```

- Resultado:** (101, 'Ana', 16.5).
- c) Verificar **len(campos) == 3**. Depois, validar as conversões e as regras dos dados. Ter três campos não garante que o número e a nota sejam válidos.

5 VALIDAR UM IDENTIFICADOR

- a) Ana12: válido.
12Ana: inválido, começa por algarismo.
Ana-1: inválido, contém hífen.
Ana: inválido, só tem três caracteres.
- b) Uma string vazia não tem posição 0. A verificação do comprimento permite rejeitá-la antes desse acesso.
- c) **strip()** remove espaços nas extremidades; **isalnum()** testa letras/algarismos; **texto[0].isalpha()** testa a primeira letra.

6 DESAFIO INTEGRADOR

```
def contar_palavras(texto):
    palavras = texto.lower().split()
    return len(palavras)
```

- b) 'Python é útil' produz ['python', 'é', 'útil'] e devolve **3**. Uma string só com espaços produz [] e devolve **0**.
- c) Não. A pontuação permanece nos tokens. Por exemplo, 'Olá, mundo!'.split() produz ['Olá,', 'mundo!'].